

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет технологий искусственного интеллекта

Дисциплина:

Web-разработка: Frontend

Отчет

Лабораторная работа 2:
взаимодействие с внешним API

Выполнил:

Александров Артем

J3314

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Теперь нужно привязать то, что делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию

Ход работы

1. Развертывание локального мокового бекэнда

- Спроектирована структура базы данных в файле «db.json», включающая три ключевые сущности:
 - **users** — учетные записи (идентификатор id, name, email, password);
 - **accounts** — привязанные банковские счета и карты с балансом и указанием валюты;
 - **transactions** — финансовые операции с привязкой к владельцу через внешний ключ userId, указанием типа, категории, суммы и даты в формате ISO.
- Запуск виртуального REST-сервера осуществлен через команду «`npx json-server --watch db.json --port 3000`».

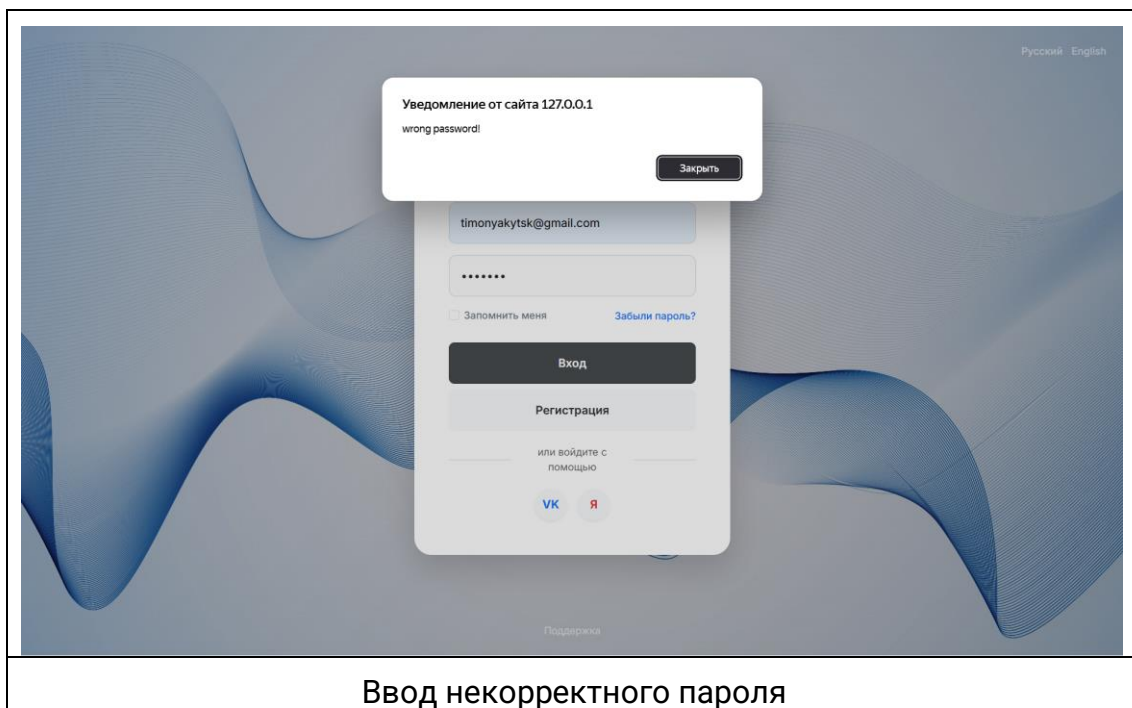
2. Архитектура сетевого слоя

- Для изоляции сетевых вызовов от логики пользовательского интерфейса создан модуль **api.js**.
- Запросы реализованы с помощью стандартного **Fetch API** и асинхронного синтаксиса **async/await** с обязательной обработкой исключений в блоках **try-catch**
- Разработаны функции-обертки:
 - `getUserByEmail(email)` — выполнение GET-запроса с URL-кодированием (`encodeURIComponent`) для безопасного поиска пользователя по параметрам;

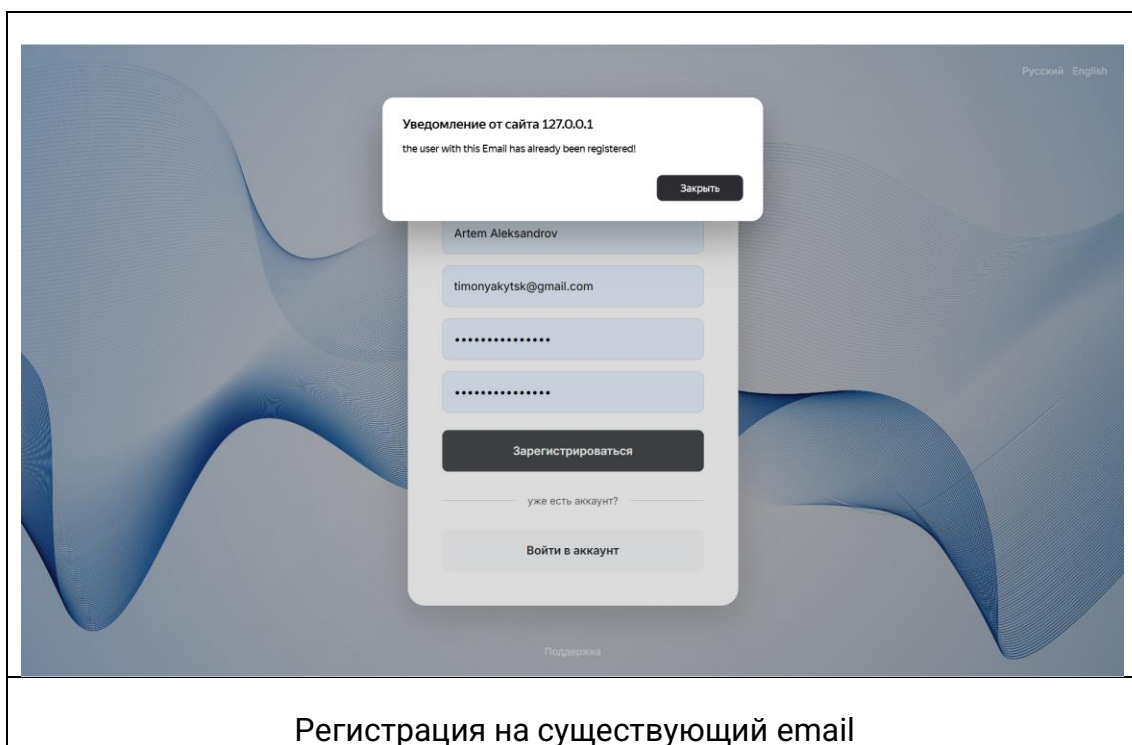
- `createUser(userData)` — отправка POST-запроса с заголовком и сериализацией объекта в строку через;
- `getTransactions(userId)` — загрузка транзакций с поддержкой тернарного формирования URL для фильтрации по конкретному `userId`;
- `createTransaction(data)` — добавление новой операции на сервер.

3. Система аутентификации и клиентские сессии

- Менеджер сессий (**auth.js**): Состояние текущего вошедшего пользователя сохраняется в хранилище браузера *localStorage* под ключом *'currentUser'*.
 - Функция *checkAuth()* выполняет роль клиентского барьера: при попытке открыть *dashboard.html* или *search.html* без сохраненного объекта в *localStorage* пользователя перенаправляет на экран авторизации *index.html*.
 - Функция *logout()* очищает хранилище и переводит пользователя на стартовую страницу.
- Страница входа (**login.js**): Обработчик формы валидирует совпадение пароля с полученной с сервера записью и инициализирует сессию.
- Страница регистрации (**register.js**): проверяет совпадение полей пароля, предотвращает дублирование email через *getUserByEmail()*, отправляет POST-запрос на регистрацию и сразу выполняет вход.



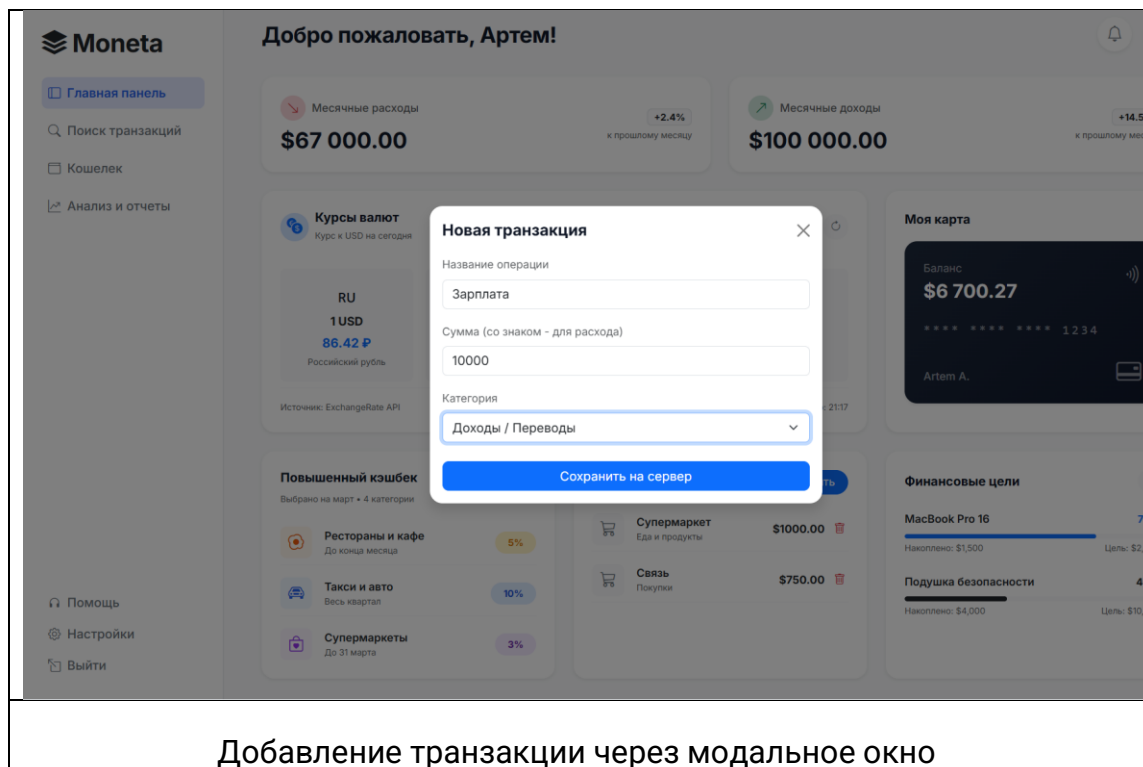
Ввод некорректного пароля



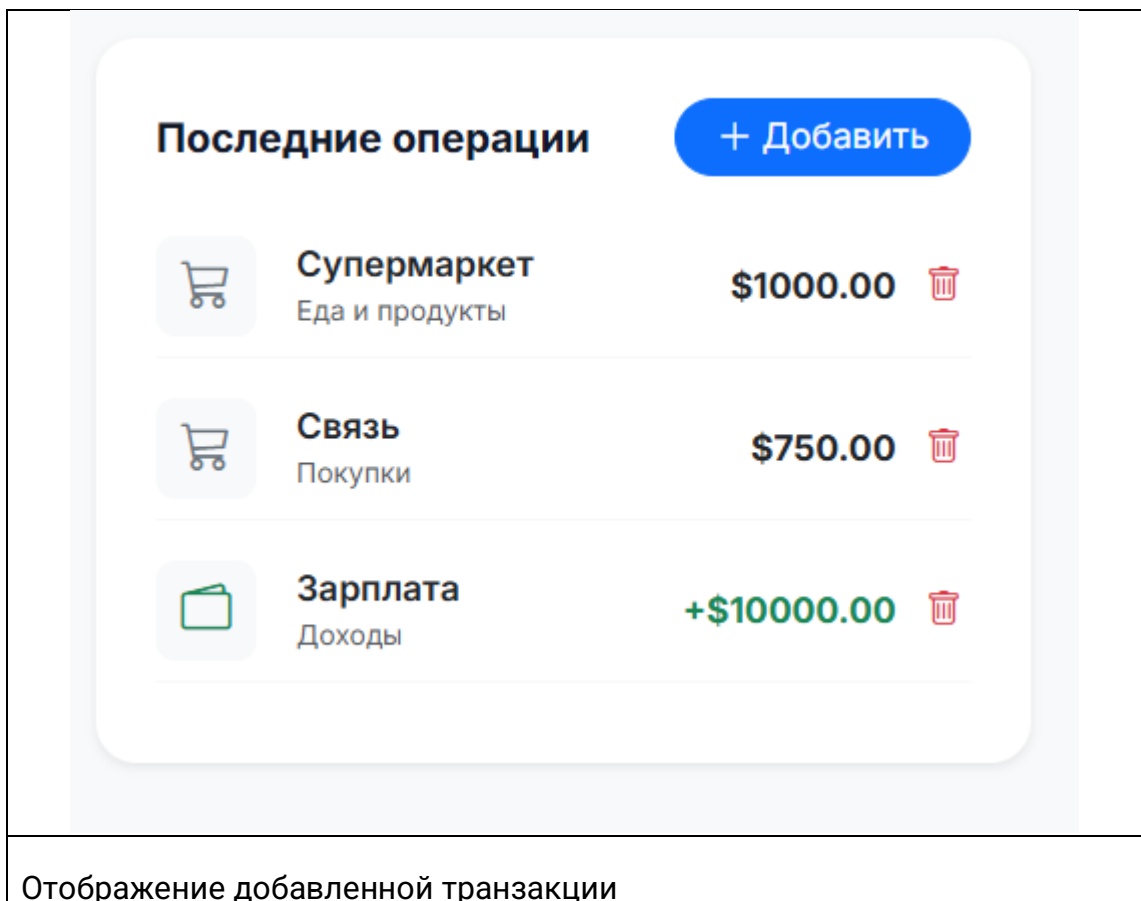
Регистрация на существующий email

4. Динамическое управление Дашбордом

- **Инициализация экрана:** динамически подставляет имя пользователя в приветственный заголовок.
- **Чтение и рендеринг:** функция `loadAndRenderTransactions(userId)` формирует динамическую разметку ленты операций, форматируя суммы и встраивая иконки категорий.
- **Создание операции:** в разметку `dashboard.html` интегрировано модальное окно Bootstrap. По отправке формы создается объект, отправляется на сервер, а список транзакций обновляется без перезагрузки экрана.
- **Удаление операции:** к каждой транзакции привязана кнопка удаления, которая после подтверждения пользователем выполняет прямой запрос `fetch(`${API_URL}/transactions/${txId}', { method: 'DELETE' })` и повторно синхронизирует отображение с базой данных.



Добавление транзакции через модальное окно



5. Взаимодействие с внешним открытым API

- На дашборде реализован блок «**Курсы валют**», заменивший статическую заглушку из первой лабораторной работы.
- Модуль выполняет асинхронный GET-запрос к публичному API
- Отображение и отказоустойчивость:
 - Из входящего JSON-объекта извлекаются котировки для целевых валют (RUB, EUR, CNY, GBP) и округляются до 2 знаков после запятой
 - Добавлена визуализация процесса загрузки через спиннер Bootstrap и отметка точного времени обновления данных
 - При возникновении сетевого сбоя или недоступности API интерфейс выводит информативный блок с предупреждением

- Реализована кнопка ручного обновления котировок по клику на кнопку загрузки

**Курсы валют**
Курс к USD на сегодня

RU

1 USD

86.42 ₺

Российский рубль

EU

1 USD

0.86 €

Евро

CN

1 USD

6.73 ¥

Китайский юань

GB

1 USD

0.74 £

Фунт стерлингов

Источник: ExchangeRate API

Обновлено: 21:20

**Курсы валют**
Курс к USD на сегодня

RU

1 USD

86.42 ₺

Российский рубль

EU

1 USD

0.86 €

Евро

CN

1 USD

6.73 ¥

Китайский юань

GB

1 USD

0.74 £

Фунт стерлингов

Источник: ExchangeRate API

Обновлено: 21:31

Обновление курса валют (см. время)

Выводы

В ходе выполнения лабораторной работы закреплены практические навыки асинхронного программирования на JavaScript с применением интерфейса Fetch API, промисов и операторов `async/await`.

Освоена работа с локальным моковым сервером JSON-Server, позволившая смоделировать реальное поведение клиент-серверной архитектуры с поддержкой HTTP-методов GET, POST и DELETE

Построена модульная структура клиентского кода с разделением зон ответственности: изоляция сетевых запросов (api.js), логика авторизации и сессий (auth.js), управление интерфейсом (dashboard.js) и интеграция стороннего REST-сервиса (currency.js)